基于大模型的软件测试使用记录

1 黑盒测试使用记录

1.1 任务一：测试计划

先向模型说明课程作业要求，并请模型判断当前应优先完成的任务。随后继续要求模型检查《黑盒测试计划》是否有多余内容，并按更贴合作业要求的方式调整文档结构。

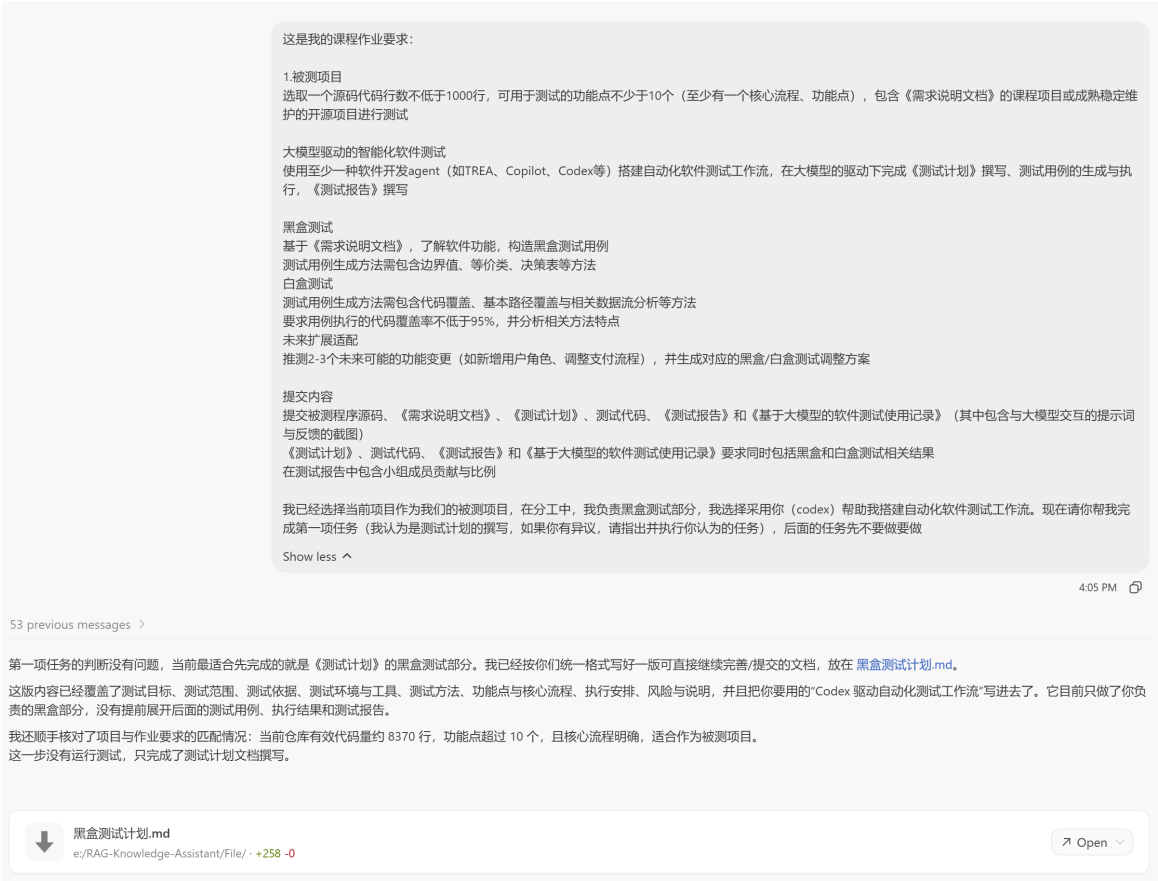


图 1: 任务一开始：提供课程作业要求，并请模型先完成黑盒测试计划

图 2: 任务一改进：继续检查测试计划是否存在多余内容并进行精简

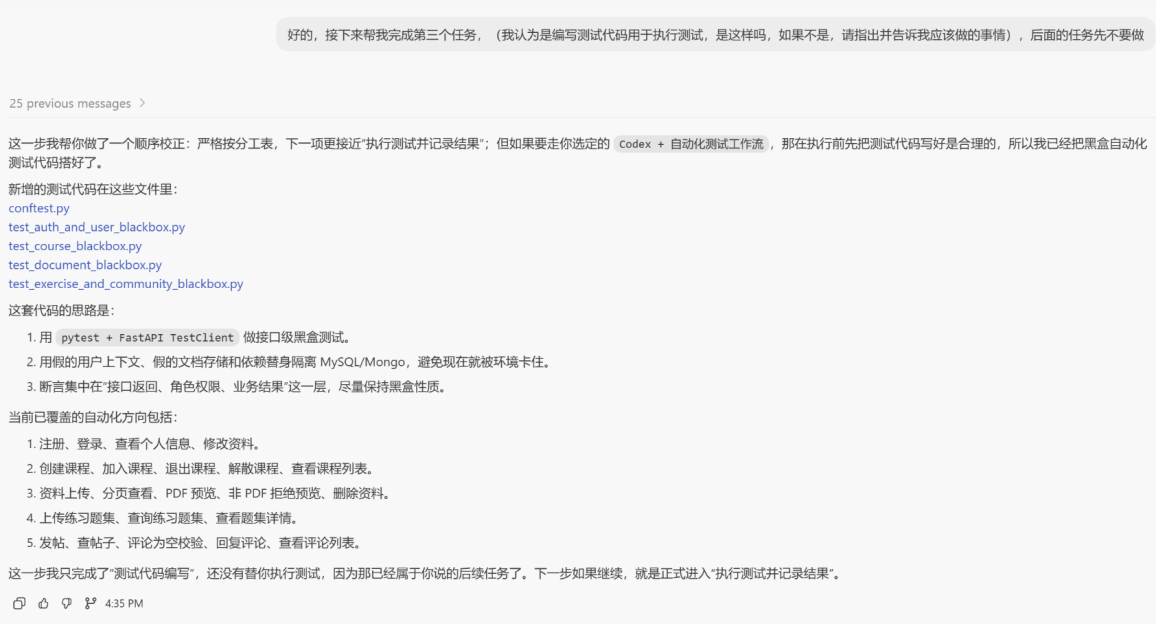
1.2 任务二：测试用例

继续让模型完成黑盒测试用例设计。模型根据前面的任务顺序判断，继续生成黑盒测试用例，并说明已经覆盖用户、课程、资料、练习题、社区和核心流程等模块。

图 3: 任务二：构造黑盒测试用例

1.3 任务三：生成并执行黑盒测试

这一阶段主要涉及“第三项任务到底是什么”“是否要先写测试代码”“自动化测试是否偏离黑盒测试”“是否需要少量人工补充”等问题。从编写自动化测试代码，到执行测试，再到继续调整测试结构和说明黑盒边界，相关交互见下列截图。



好的，接下来帮我完成第三个任务，（我认为是编写测试代码用于执行测试，是这样吗，如果不是，请指出并告诉我应该做的事情），后面的任务先不要做

25 previous messages

这一步我帮你做了一个顺序校正：严格按分工表，下一项更接近“执行测试并记录结果”；但如果要走你选定的 `Codex + 自动化测试工作流`，那在执行前先把测试代码写好是合理的，所以我已经把黑盒自动化测试代码搭好了。

新增的测试代码在这些文件里：

conftest.py
test_auth_and_user_blackbox.py
test_course_blackbox.py
test_document_blackbox.py
test_exercise_and_community_blackbox.py

这套代码的思路是：

1. 用 `pytest + FastAPI TestClient` 做接口级黑盒测试。
2. 用假的用户上下文、假的文档存储和依赖替身隔离 MySQL/Mongo，避免现在就被环境卡住。
3. 断言集中在“接口返回、角色权限、业务结果”这一层，尽量保持黑盒性质。

当前已覆盖的自动化方向包括：

1. 注册、登录、查看个人信息、修改资料。
2. 创建课程、加入课程、退出课程、解散课程、查看课程列表。
3. 资料上传、分页查看、PDF 预览、非 PDF 拒绝预览、删除资料。
4. 上传练习题集、查询练习题集、查看题集详情。
5. 发帖、查帖子、评论为空校验、回复评论、查看评论列表。

这一步我只完成了“测试代码编写”，还没有替你执行测试，因为那已经属于你说的后续任务了。下一步如果继续，就是正式进入“执行测试并记录结果”。

4:35 PM

图 4: 任务三开始：初次确认第三项任务应围绕生成并执行黑盒测试展开

我要纠正一点，分工文档不是严格标准，这个只是一个参考分工的东西，严格标准是我最开始给你的课程作业要求，那个才是我真正要做的事情，所以你要判断好第三个任务是什么

你这个纠正是对的。后面我会以你最开始给出的课程作业要求为准，不再把分工文档当成“严格顺序”。

按课程作业要求，围绕黑盒测试这条线，合理的任务顺序更应该是：

1. 编写《测试计划》
2. 构造黑盒测试用例
3. 生成并执行测试
4. 记录结果并撰写《测试报告》
5. 整理《基于大模型的软件测试使用记录》

所以你现在说的“第三个任务”，更准确地讲不是单纯“编写测试代码”，而是：

`生成并执行黑盒测试`

其中“编写测试代码”只是这一步里的一个子任务，不是最终目标。
也就是说，我上一步做的事情属于第三个任务的前半段：先把自动化测试代码搭起来，为执行测试做准备；但第三个任务本身还没完成，因为还没有真正执行测试并产出结果。

如果你同意，下一步我就按真正的第三个任务继续：

1. 先检查这套测试代码是否可运行
2. 修正不能运行的地方
3. 实际执行黑盒测试
4. 整理测试结果记录

你这次说“后面的任务先不要做”，那我建议我们现在就只做第三个任务本体的下一步，也就是：`执行并调通这些黑盒测试`。

4:35 PM

图 5: 任务三调整：围绕“测试代码为主、必要时少量人工”的要求调整测试方案

好的，按照你说的，继续完成第三个任务

26 previous messages

第三个任务已经推进到“生成并执行黑盒测试”这一步了，而且当前这套自动化黑盒测试已经成功跑通。

我完成了这些内容：

1. 搭建了黑盒自动化测试基座 conftest.py，用 `FastAPI TestClient + pytest` 执行接口级黑盒测试。
2. 编写了分模块测试代码：
   tests
3. 补了 pytest.ini，把 pytest 固定到 `tests/` 目录，避免缓存目录干扰。
4. 实际执行了测试，并把执行中暴露出来的断言格式和临时目录问题修正掉了。

本次执行命令是：

powershell

```
D:\miniconda3\envs\rag\python.exe -m pytest -q
```

执行结果：

text

```
28 passed, 5 warnings in 0.33s
```

这 28 条自动化测试目前覆盖了：

1. 注册、登录、查看个人信息、修改资料
2. 创建课程、加入课程、退出课程、解散课程、查看课程列表
3. 资料上传、查看、PDF 预览、非 PDF 拒绝预览、删除资料
4. 练习题上传、查询、查看详情
5. 发帖、查帖子、评论校验、回复、查看评论

说明一下，这一步我已经完成了第三个任务里“生成并执行测试”的核心部分，但还没有开始第四个任务，也就是“整理执行结果并撰写测试报告”。如果你继续，我下一步就帮你把这次执行结果整理成可直接放进作业的测试执行记录和《测试报告》内容。

4:40 PM

4 files changed +38 -8 Undo Review

Code/backend/pytest.ini +4 -0

Code/backend/tests/test_auth_and_user_blackbox.py +3 -2

Code/backend/tests/test_document_blackbox.py +28 -4

Code/backend/tests/test_exercise_and_community_blackbox.py +3 -2

图 6: 任务三继续：继续第三项任务，模型搭建自动化黑盒测试代码并给出执行结果

你确定你用pytest编写的是黑盒测试吗，有没有偏离黑盒测试？pytest是一个好选择吗？以及，你给我的人工执行清单有没有偏离黑盒测试？对于上面的问题，进行思考检查之后回答我

结论先说：

1. `pytest` 本身可以用于黑盒测试
2. 但我给你写的这套自动化测试，不是“最纯粹的黑盒测试”，更准确地说是“接口级黑盒测试为主，带一点灰盒色彩”
3. 人工执行清单整体上没有偏离黑盒测试，反而比自动化部分更接近纯黑盒

关于“pytest 编写的是不是黑盒测试”
`pytest` 只是测试框架，不决定测试一定是黑盒、白盒还是灰盒。
关键看“测的对象”和“断言依据”：

1. 如果通过公开接口、页面、输入输出结果来验证功能，不依赖内部实现细节，那就是黑盒思路。
2. 如果直接针对内部函数、内部路径、内部状态分支做验证，那就更偏白盒。
3. 如果表面上测接口，但为了跑起来大量替换内部依赖、利用源码结构设计测试，那通常就是灰盒。

你现在这套自动化测试的优点是：

1. 入口是 HTTP API，不是直接测 repository/service 私有逻辑。
2. 断言主要看状态码、返回体、权限控制、业务结果。
3. 从“被测对象”看，确实是面向外部接口的。

但它不够“纯黑盒”的地方也要诚实说：

1. 我用了 `monkeypatch` 去替换控制器依赖和服务函数。
2. 我用了 `dependency_overrides` 、假的 DAO、假的用户上下文。
3. 我是基于源码结构知道该 patch 哪些对象，才让测试隔离运行。

所以严格说，这套测试不是“完全纯黑盒”，而是：

`基于公开 API 的接口级黑盒测试 + 为了自动化落地而引入的灰盒实现手段`

这个说法最准确，也最诚实。

pytest 是不是一个好选择
是，`pytest` 是一个好选择，而且对你这个作业是合理的。原因是：

1. 你的项目后端是 FastAPI， `pytest + TestClient` 是很常见的自动化方案。
2. 课程作业要求的是“大模型驱动的自动化软件测试工作流”， `pytest` 很适合承接自动化执行和结果整理。
3. 它容易产出可运行、可重复执行、可展示的测试成果。

但也要区分层次：

1. 如果目标是“后端接口级黑盒测试”， `pytest` 很合适。
2. 如果目标是“严格端到端的用户视角黑盒测试”，更合适的是 `Playwright` 、 `Selenium` 这类浏览器自动化工具。
3. 如果目标是“对真实部署系统做最纯粹的黑盒接口测试”，也可以用 `pytest + requests` 直接打运行中的服务，而不是像我现在这样 patch 内部依赖。

所以对你这份作业来说，我建议表述成：

`自动化黑盒测试采用 pytest 对系统公开接口进行测试执行；人工测试补充页面交互与用户可见行为验证。`

这样就稳。

我给你的自动化测试有没有偏离黑盒测试
有一点偏离，但不算严重失真。
更准确地说，它偏到了“灰盒辅助的黑盒自动化”。

如果老师非常严格地咬“黑盒”这个词，你可以这样解释：

1. 测试目标仍然是系统公开接口行为，不是内部算法覆盖。
2. 测试判定依据是需求和接口输出，不是源码路径。
3. 为了降低环境搭建成本、提高自动化可执行性，测试实现中使用了依赖替身。

这在课程作业里通常是能接受的，只要你别把它吹成“纯端到端黑盒”就行。

人工执行清单有没有偏离黑盒测试
整体没有，而且它比自动化部分更标准地贴近黑盒测试。

因为人工清单主要看的是：

1. 页面提示
2. 表单校验
3. 页面跳转
4. 列表刷新
5. PDF 预览
6. 下载行为
7. 评论回复定位
8. 核心流程中的用户可见结果

这些全都是从用户视角出发，不依赖内部实现，所以这是很典型的黑盒测试内容。

最实在的建议
如果你要把这份作业说得更严谨，我建议你后面统一用这组表述：

1. `自动化部分` ： 接口级黑盒测试，采用 `pytest` 执行。
2. `人工部分` ： 页面交互级黑盒测试。
3. `整体方式` ： 自动化接口测试 + 人工页面验证结合。

不要把当前自动化部分说成：
`完全纯黑盒端到端测试`

那样会说得太满。

如果你愿意，我下一步可以继续帮你做一件很重要的事：
`把当前自动化测试的性质重新整理成一段可直接写进测试报告的方法说明`
这样你后面交作业时表述会非常稳。

5:11 PM

图 7: 任务三检验：继续检验 pytest、黑盒测试边界与人工测试定位

不要关注于改解释，而是思考是否需要改测试代码和人工测试。如果不够符合作业要求，则进行修改；如果符合，则不需要。且黑盒测试应当以测试代码为主，如果需要，可引入少量人工。

26 previous messages

我检查后的结论是：`需要改自动化测试代码，但不需要大改人工测试清单`。我已经完成了这次修改。

现在的判断更明确了：

1. 之前那版自动化测试"能用"，但测试函数里大量逐条 `monkeypatch` 控制器/服务返回值，偏接口外壳验证，黑盒味道不够纯。
2. 按你要求"黑盒测试应以测试代码为主，必要时少量人工"，那自动化部分应该更像"通过公开接口驱动一套业务状态"，而不是每条测试都手工伪造结果。
3. 所以我已经把自动化测试改成了更接近黑盒接口测试的结构：
   - 在 conftest.py 里统一构建了内存态假后端
   - 在夹具层一次性接管持久化边界与用户上下文
   - 测试函数本身只做"切换用户、发请求、断言结果"

这次修改后的效果是：

1. 测试代码依然以 `pytest` 为主，没有改成解释性文字补丁。
2. 自动化测试仍然覆盖注册、登录、个人信息、课程、资料、练习题、社区、核心流程。
3. 测试函数比之前更少依赖对单个控制器函数的人为返回值伪造，更符合"以接口行为为中心"的黑盒风格。

我保留了人工执行清单，原因是：

1. 它本身没有偏离黑盒测试。
2. 它主要补页面交互、PDF 预览、下载行为、定位跳转这些自动化接口测试不擅长覆盖的点。
3. 现在它就是"少量人工补充"，不是主体。

最新执行结果：

```text
36 passed, 5 warnings in 0.44s
```

执行命令：

```powershell
D:\miniconda3\envs\rag\python.exe -m pytest -q
```

所以现在可以比较稳地说：

1. 黑盒测试主体已经是"测试代码为主"
2. 人工测试只是补充
3. 相比之前，这一版更符合你的作业要求

如果你继续下一步，我建议就进入"整理执行结果并准备测试报告材料"。

5:28 PM

图 8: 任务三补充：补充说明课程作业要求与任务顺序之间的关系

1.4 任务四：执行记录与测试报告

这一阶段主要是整理测试执行记录、编写测试报告，并继续检查测试计划与测试报告是否存在冗余内容。模型根据执行结果汇总自动化测试与人工测试数据，并继续对文档结构和表述方式进行修改。

我已经补充完了人工测试。接下来帮我完成第四个任务，（我认为是整理测试执行记录、编写测试报告，是这样吗，如果不是，请指出并告诉我应该做的事情），完成之后是否这部分就结束了？

7 previous messages

第四个任务你的判断基本是对的。对你负责的黑盒测试部分来说，这一步就是：

1. 整理测试执行记录
2. 编写黑盒测试报告

我已经帮你完成了两份文档：
黑盒测试执行记录.md
黑盒测试报告.md

这两份内容已经整合了：

1. 自动化黑盒测试结果：`36 passed`
2. 人工黑盒测试结果：`10/10 通过`
3. 黑盒测试总执行数：`46 项`
4. 主要覆盖模块、执行方式、结果统计、问题分析和测试结论

关于"完成之后这部分是否就结束了"，我的判断是：

1. 如果只看你负责的黑盒测试部分，基本可以算结束了。
2. 你这部分已经具备了：
   - 黑盒测试计划
   - 黑盒测试用例
   - 黑盒自动化测试代码
   - 黑盒人工测试执行清单
   - 黑盒测试执行记录
   - 黑盒测试报告

但从整个课程作业总要求看，还差最后一类材料不属于"黑盒测试本体"，那就是：

1. 《基于大模型的软件测试使用记录》
2. 如果你们最终提交的是整组总报告，还需要和白盒测试部分汇总整合

所以准确地说：

1. 你的黑盒测试主体工作已经完成
2. 你个人这部分还剩一个收尾材料：大模型使用记录
3. 如果你愿意，下一步最自然的就是我帮你做这份《基于大模型的软件测试使用记录》

7:14 PM

2 files changed +251 -0 Undo Review

e:/RAG-Knowledge-Assistant/File/黑盒测试执行记录.md +108 -0

e:/RAG-Knowledge-Assistant/File/黑盒测试报告.md +143 -0

图 9: 任务四开始：整理测试执行记录并编写测试报告

由于测试计划和测试报告是我需要交的两份材料，你需要再次仔细检查，是否写了多余无用冗杂的部分需要删除，是否存在写的不好不对的地方需要修改。文档结构是否清晰，注意，核心约束就是作业要求，分工文档中的统一格式不需要遵循，你应当按照最符合作业要求，最适合的方式，撰写和组织这两份文档。
此时不需要写贡献比例。

18 previous messages

我已经重新检查并精修了这两份材料，现在它们更贴合作业要求，也更适合直接提交：

黑盒测试计划.md
黑盒测试报告.md

这次主要做了这些调整：

1. 不再机械套用分工文档的统一格式，而是按课程作业真正需要的内容来组织结构。
2. 删掉了容易显得冗余或"为凑格式而写"的表达，保留对作业有支撑作用的内容。
3. 把"测试方式、范围、方法、执行结果、问题分析、结论"整理得更清晰，逻辑更顺。
4. 去掉了你这次明确不要写的贡献比例。
5. 修正了报告里过于细碎的表述，比如测试耗时这种没必要写得太死的数据。

我现在的判断是：

1. 《黑盒测试计划》结构清晰，重点明确，能够说明你为什么这样测、测什么、怎么测。
2. 《黑盒测试报告》结果、分析和结论都比较完整，没有明显多余段落。
3. 这两份文档已经可以作为你黑盒测试部分的正式提交材料使用。

所以，这一部分现在基本收束好了。你如果下一步要我继续，我就可以进入你说的后续材料，比如《基于大模型的软件测试使用记录》。

7:28 PM

图 10: 任务四改进：继续检查并精修测试计划与测试报告

2 白盒测试使用记录

预留给后续白盒测试成员补充截图和简要说明。